CausalFill
BENCH

A BENCHMARK FOR TOOL-USING AGENTS

Recover the missing.
Prove the result.

Choose what to observe.
Make the completed procedure work.

MASKED TASKS | ACTIVE EVIDENCE | CHECKED OUTCOMES

Concept + direction: Phillip A. Holland Jr. / AI-assisted research



OBSERVATION / COMPLETION / VERIFICATION

Signal Repair

Evidence before irreversible action.

A source-bound release report on globally bounded sensor corruption, exact finite cost certificates, generic formal guarantees and the limits revealed by development evaluation.

15	9	Passed
Task configurations	**Conceptual domains**	**Release verification**
Implemented suite	Shared public protocol	Consolidated source record

Concept and direction: Phillip A. Holland Jr.
Implementation and research synthesis: AI-assisted.

01 / RELEASE EVIDENCE

Inspectable evidence

The release combines executable masked tasks, budgeted observations and acceptance rules. Its strongest claim is about what has been checked under explicit models.

Measured

316

Unit and integration tests

0 failures; 0 errors

3,584

Independent optimum comparisons

Finite models and objectives

1,280

Certificate mutations rejected

Specified mutation families

6 / 6

Assistant pilot episodes passed

Small development pilot

Three evidence classes

Label	What it supports
Measured	An observed test, replay, episode or implementation property in the recorded configuration.
Formally checked	A named theorem accepted by the listed proof-checking configurations, under its stated assumptions.
Proposed	A future experiment, generalization claim or research goal that this release has not established.

INTERPRETATION

These counts use different units and must not be added into an intelligence score. The report separates scripted controls, finite certificate audits, formal theorem checks and a small assistant pilot.

Source: reports/v051/release-verification.json; exact record hash on page 12.

02 / BENCHMARK CONTRACT

One protocol, several kinds of missing information

15 runnable configurations across 9 conceptual domains. Signal Repair adds robust_sensing to the diagnosis and decision domain.

Domain	Configurations	What must be completed
Routing words	words, ambiguity	A condition or a justified abstention
Executable workflow	workflow	An expression meeting a restricted semantic contract
Grid induction	grid	A transformation inferred from examples and experiments
Procurement	vending, vending_adaptive	An operational purchasing procedure
Robotics in simulation	robotics, robotics_barrier, robotics_vision	A geometry-dependent plan or controlled image-to-action choice
Mathematical state	latent, latent_sparse	A finite-field operator or sparse structure
Proof completion	proof	A proof of a fixed claim with a consistency witness
Diagnosis and decision	diagnosis, robust_sensing	A justified state or robust terminal action
Controlled vision	visual	Pixel-derived spatial facts and an executable choice

COMMON CONTRACT

A public episode exposes the missing component and permitted tools. The agent spends a declared budget, submits a typed completion, and faces evaluator-owned acceptance checks.

The computer interface is a virtual workspace with typed tools. The image and robotics tasks use controlled renderings and small simulations. Their results do not establish real desktop competence or physical robot transfer.

Source: README.md; docs/protocol.md; docs/scoring.md.

03 / SIGNAL REPAIR

A shared fault cap changes what evidence means

A fixed hidden world determines truthful binary sensor replies. Up to k replies may be flipped across the whole episode, and each listed query may be used once.

INFORMATION STATE

For each world, retain its discrepancy count and the remaining queries. A world is feasible exactly when its discrepancy count is at most k. A set of surviving world IDs alone loses necessary fault-history information.

```
d_h(w) = number of replies disagreeing with world w
possible(h) = { w : d_h(w) <= k }
```

Worked example: one possible flipped reply

Two worlds require exclusive left/right actions. Three unit-cost queries all truthfully report 0 in the left world and 1 in the right world. A terminal action costs 2. The example illustrates the published model; it is not a model-evaluation episode.

Query	Left truth	Right truth	Observed
sensor_a	0	1	1
sensor_b	0	1	0
sensor_c	0	1	0

The left world needs one flip; the right world needs two. With k = 1, sensor_b and sensor_c are distinct observed contradictions that exclude the unsafe right world. The action can now be justified.

EXACT EXAMPLE COST

A policy can stop after two equal readings. Its worst-case action cost is 3 queries + 2 for the terminal action = 5. Identification alone costs 3. These are analytic fixture values, not a frontier score.

Source: docs/robust-decisions.md; docs/robust-research.md.

04 / CERTIFICATE SCOPE

An attaining policy is only half the argument

The release separates exact optimization, independent certificate checking, fixed-world policy replay and observed evidence checking.

Component	Representation	Claim checked
Optimizer	Disjoint world bitmasks at each fault-debt layer	Proposes state values and attaining choices
Exact certificate checker	Integer discrepancy tuples and explicit world sets	All available alternatives and feasible observation children
Policy replay	A fixed world and the faults actually consumed	Successful execution and an upper cost bound
Completion evidence	Distinct accepted observations from the transcript	At least k+1 contradictions for each unsafe world

The exact finite objective

At a nonempty feasible state, consider every action that succeeds in every remaining world and every unasked query. A query contributes its cost plus the worst feasible child value. The minimum is the exact guaranteed cost. An impossible continuation remains distinct from zero and from a missing row.

```
V(state) = min(safe terminal costs,
               query cost + max(child values))
```

WHY THE DISTINCTION MATTERS

A replayed policy may be valid but needlessly expensive. Exactness requires checking every legal alternative, including queries the attaining policy does not choose. The zero-cost safe-action shortcut follows from nonnegative costs.

The Python finite-state certificate is not itself a Lean proof of the optimizer or of the runtime. The separately checked generic Lean statements have a narrower scope, described on page 8.

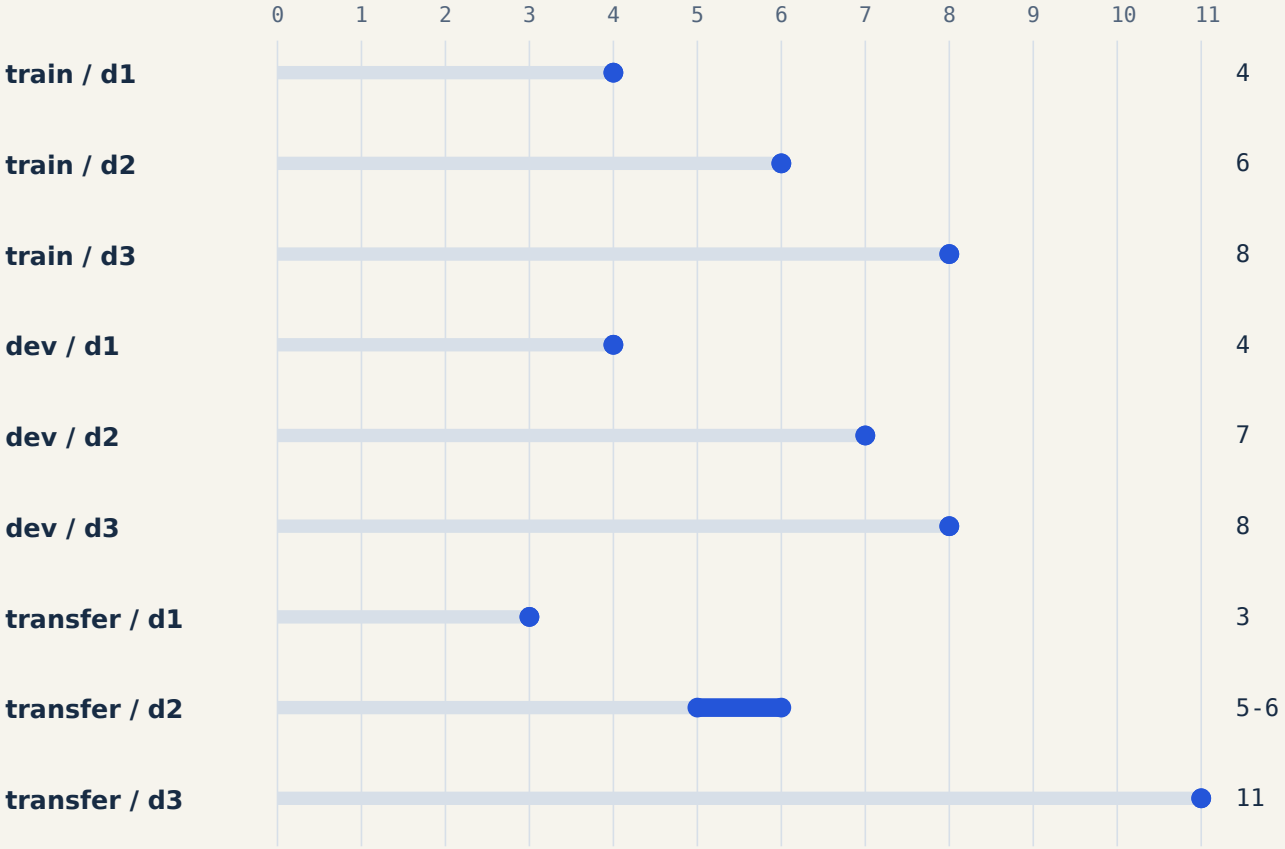
Source: docs/robust-decisions.md; formal scope recorded in release-verification.json.

05 / EXACT COST PROFILES

A budget grounded in attainable cost

Source-derived costs for the recorded runtime profiles. These synthetic operational units measure permitted sensing and commitment, not tokens, dollars or elapsed time.

Measured



Certified operational cost: queries plus the terminal action. Dots show the minimum and maximum across recorded seeds, not a confidence interval.

27 profile records cover 2,652 fixed-world/fault cases. The runtime checks recorded 27 attained profile bounds.

READ THE UNITS

The certified operational budget covers domain probes and the terminal action. Public-file reads and fill-envelope work may have separate tool costs. A lower cost in one structural family is not evidence that it is easier for a language model.

Source: release-verification.json / robust_sensing.runtime.profiles and checks.runtime_bound_attained.

06 / INDEPENDENT AUDIT

Stress tests aimed at concrete failure modes

Different representations and analytic fixtures challenge the implementation from several directions. Coverage counts remain tied to their own units.

Measured

1,536
Complete tiny models
2 objectives per model

256
Additional random models
Deterministic recorded generation

1,280
Mutant artifacts rejected
Targeted certificate alterations

2,652
Executed runtime cases
World/fault assignments in profiles

Audit	Recorded count	Interpretation
Independent optima	3,584	Optimizer versus independently represented exact oracle
Policy replay	1,644	Fixed-world corruption histories
Analytic formula cases	180	Weighted cube family with derived cost
Oracle states	48,842	Finite search coverage; not independent tasks
Certificate state occurrences	74,522	Occurrences across checks; repeats are possible

Mutations target stale model digests, wrong optima, duplicate states, missing roots and objective confusion. Additional checks exercise cost scaling, renaming and bit polarity, corruption monotonicity, and query-addition monotonicity.

Source: reports/robust-stress-v0.5.1/result.json via the consolidated record. Tiny-corpus scope: complete 2-world/2-query/2-action binary tables, k=0..2, two explicit cost profiles

07 / FORMAL ASSURANCE

Generic guarantees with an explicit boundary

The formal gate concerns declared Lean theorem statements. It is separate from the finite Python optimal-cost certificate and the executed task runtime.

Formally checked

14	6	6
Audited theorem roots	**Checker configurations**	**Valid kernel checks**
Required root inventory	Recorded proof-checking backends	Accepted exported proof artifacts

What the generic statements cover

The bounded-fault development formalizes discrepancy accounting, preservation of a true world within the global fault cap, permanent exclusion after too many contradictions, occurrence-supported witnesses, and justified actions over overlapping success regions.

These claims assume one fixed true world in the published world list and an actual corruption count within the cap. The occurrence-based Lean witness and the runtime's once-only experiment-ID witness still require an explicit correspondence boundary.

NOT ESTABLISHED BY THIS FORMAL GATE

The Bellman optimizer's exact result, general Python/Lean equivalence, sensor-table correctness, fault bounds on physical hardware, physical execution and frontier-model difficulty remain outside these generic theorem statements.

Recorded allowed axioms: Classical.choice, Quot.sound, propext

The checker configurations share portions of the toolchain. Their count does not mean wholly independent proof stacks.

Negative kernel checks: 6. Statement-binding and negative controls are retained in the formal source record; successful proof checking cannot substitute for checking that the intended statement was proved.

Source: formal/verification-v051-fault/run-02/results.json; formal/verify_bounded_fault.py; docs/robust-research.md.

08 / ASSISTANT PILOT

What the pilot establishes

The pilot records public-interface attempts separately from scripted development controls. Its sample size and isolation conditions must travel with the success count.

Measured

6 / 6

Pilot episodes passed

Observed development attempts

Yes

Core source stable

Fingerprint checked during pilot

Episode / result	Queries	Operation cost	Worlds left	Assigned faults	Observed faults
train d3 / pass	7	8	1	1	1
dev d2 / pass	6	7	1	1	1
dev d3 / pass	7	8	1	1	1
transfer d1 / pass	2	3	6	1	1
transfer d2 / pass	0	5	8	1	0
transfer d3 / pass	7	9	1	2	1

Assigned fault slots need not be queried. A bypass can succeed without observing a corrupted reply; action sufficiency can also leave several worlds unresolved.

HARDNESS BOUNDARY

All recorded pilot episodes succeeded. This is evidence against presenting this small pilot set as currently unsolved or frontier-hard. It motivates independent structural holdouts and stronger calibrated tests.

OS-enforced solver isolation: No. Model inference telemetry available: No. Wall time is recorded execution time and must not be presented as inference latency.

Source: release-verification.json / assistant_pilot; the source record retains tool cost and per-episode details.

09 / SCRIPTED CONTROLS

Controls and ablations

These are restricted, task-aware development scripts. Their success counts measure behavior under the task contract, not trained-model capability.

Measured

Recorded control episodes: 4,050. The table separates answerable completion from correct handling of deliberately unanswerable cases.

Policy	Episodes	Answerable success	Correct unanswerable abstention
Task-aware evidence reference	1,350	1,260 / 1,260	90 / 90
Blind control	1,350	0 / 1,260	0 / 90
Abstention control	1,350	0 / 1,260	90 / 90

Signal Repair ablations

Restricted script	Strict success	Actual operation succeeded	Justified at commit
Assume no corruption	0 / 90	79 / 90	0 / 90
Query every sensor	40 / 90	40 / 90	40 / 90

OUTCOME AND EVIDENCE DIFFER

A lucky action in the selected world does not establish that the action was justified across all compatible worlds. Conversely, requesting every sensor can spend resources that were needed for the terminal action.

The next model study should freeze settings and compare matched with-skill and without-skill conditions. Preserve valid alternative strategies and report outcome, justification, cost and abstention separately.

Source: release-verification.json / scripted_controls and robust_ablation; reports/v051/robust-ablation/summary.json; SkillsBench [10].

10 / ROBOTICS RELEVANCE

Choose a valid action before recovering every detail

Signal Repair isolates a useful decision problem: act successfully across all remaining hypotheses while preserving resources for the action.

Proposed

Controlled ingredient	Robotics interpretation	Separate validation needed
Finite worlds	Candidate object, route or fixture states	Coverage of real states and unmodeled obstacles
Shared corruption cap	A bounded number of contradictory sensor reports	Justification of the fault bound and sensor table
Overlapping success sets	Several scenes permit the same safe choice	Actuation, dynamics and geometry of each action
Paid evidence	Views, contact checks or calibration steps	Actual sensing cost and whether a probe changes state

A staged transfer experiment

First keep the symbolic model fixed and replace one observation channel with controlled rendered pixels. Next test calibration and fault extraction on recorded sensor data. Only then evaluate a bounded physical task with measured actuator limits and an independently justified hypothesis set.

Persistent degradation, continuous noise, moving objects and contact-dependent measurements need richer state models. A deterministic fault cap does not yield a probability of success. A synthetic finite certificate should therefore be treated as a component-level diagnostic.

PROPOSED EVALUATION

Measure action success, robust justification, observation cost, unnecessary identification, abstention and assumption violations. Compare these quantities before and after training on the benchmark; retain a mechanism-level holdout.

Source: docs/robust-research.md; docs/robotics-vision.md; partial-observation planning [4].

11 / REPRODUCIBILITY

The evidence belongs to an exact source state

The consolidated record binds the release's checks. The restored source and retained evidence keep their own version and provenance.

Record	Value
Release status	passed
Retained source version	0.5.0
Source inventory files verified	23,429
Unchanged / modified / new files	23,422 / 7 / 4,483
Stress source remained stable	Yes

Integrity identifiers

Core source fingerprint

`a7f0b47a3c8a26ebba2cab3dc7fdb9fa5c3ea2c211048c5d872e28e221f61c7a`

Consolidated verification JSON SHA-256

`b11767852853c6171b6235a523a046d459d5d891c1d2c60af5b5f56f95e10378`

Restored archive SHA-256

`2f8a05b6204b4bf9b353f89c883f80e16044f8e773f9b9b336a4b61c2d1d5974`

Reproduce and inspect

Start from README.md and docs/release-gates.md. The record's artifacts map points to the unit, scripted-control, stress, formal and pilot outputs. Rebuild cards with tools/build_release_cards.py and this report with tools/build_v051_report.py.

KEEP THE SCIENTIFIC BOUNDARY

Freeze the evaluator before scored solving. Preserve solver/evaluator separation, source hashes, episode traces, budgets and model settings. A public small generator is a development resource; private future families and independent replication remain necessary for broad claims.

Source: reports/v051/release-verification.json; AGENTS.md; docs/release-gates.md.

12 / PRIOR ART AND NEXT QUESTIONS

A focused contribution within an established field

The contribution is the integration of masked completion, budgeted observations, executable consequences and explicit certificates. The mathematical and evaluation ingredients have substantial prior art.

Research line	Relationship to this release
Bounded lies [1]	A fixed hidden object and a global corruption allowance are classical. The release applies them to a finite task contract.
Decision regions [2]	A valid action can require less information than exact world recovery. The new objective is worst-case rather than Bayesian.
Environment design [3]	Exact feasibility and attainable cost can gate generated tasks. They do not themselves measure a model's difficulty.
ARC and Vending [5, 6]	Human calibration, action efficiency and delayed consequences motivate the protocol. No official tasks or published scores are imported.
Axiom and OpenAI math [7, 8]	Proof-producing workflows motivate checked artifacts. The finite benchmark does not reproduce their research mathematics.

Interpret the 100D ambition precisely

The retained 100D tasks are finite-field operator-recovery problems. They do not establish physical 100-dimensional perception, a new OpenAI algorithm, or a method named '100D red teaming'. OpenAI's scientific work, public math repository and GPT-Red are distinct sources [8, 9, 12].

NEXT FALSIFIABLE CLAIM

Test whether an agent trained to preserve fault debt and submit observed witnesses improves budgeted decisions on independently authored, held-out mechanisms. Pre-register the comparison and report negative results as carefully as positive ones.

Source: docs/related-work.md; docs/math-and-100d.md; docs/robust-research.md.

13 / REFERENCES

Primary sources and local evidence

External work motivates the design; it is not evidence of CausalFill performance. The local consolidated verification record is the source for release measurements.

[1] Ulam's liar game. Osthus and Watkinson (2008). A simple solution to Ulam's liar game with one lie.
https://ems.press/content/serial-article-files/45347?nt=1

[2] Decision-region determination. Javdani et al. (2014). Near Optimal Bayesian Active Learning for Decision Making.
https://proceedings.mlr.press/v33/javdani14.html

[3] Environment design. Dennis et al. (2020). Emergent Complexity and Zero-shot Transfer via Unsupervised Environment Design.
https://proceedings.neurips.cc/paper_files/paper/2020/hash/985e9a46e10005356bbaf194249f6856-Abstract.html

[4] Partial observation. Kaelbling, Littman and Cassandra (1998). Planning and acting in partially observable stochastic domains.
https://people.csail.mit.edu/lpk/papers/aij98-pomdp.pdf

[5] ARC-AGI-3. ARC Prize Foundation (2026). ARC-AGI-3 technical report.
https://arcprize.org/media/ARC_AGI_3_Technical_Report.pdf

[6] Vending-Bench 2. Andon Labs. Persistent simulated business operations and delayed consequences.
https://andonlabs.com/evals/vending-bench-2

[7] Proof-producing research. Axiom. Proof of Concept.
https://axiommath.ai/research/proof-of-concept/

[8] OpenAI mathematics. OpenAI. Public mathematical research repository.
https://github.com/openai/math

[9] Scientific context. OpenAI. Working with academic communities to build scientific understanding.
https://openai.com/index/building-scientific-understanding-with-academic-communities/

[10] Skills evaluation. SkillsBench. Paired evaluation of skills for tool-using agents.
https://github.com/benchflow-ai/skillsbench

[11] Proof checker. Lean 4.35.0 release documentation.
https://lean-lang.org/doc/reference/latest/releases/v4.35.0/

[12] Adversarial robustness. OpenAI. GPT-Red: Unlocking Self-Improvement for Robustness.
https://openai.com/index/unlocking-self-improvement-gpt-red/

Source: Local records: reports/v051/release-verification.json; reports/robust-stress-v0.5.1/result.json; the formal and pilot artifacts named in the consolidated record.